

NCPC 2025

Presentation of solutions



October 4, 2025

NCPC 2025 solutions

Problems prepared by

- Arnar Bjarni Arnarson (Reykjavik University)
- Atli Fannar Franklín (University of Iceland)
- Nils Gustafsson (KTH Royal Institute of Technology)
- Thore Husfeldt (IT University of Copenhagen)
- Mindaugas Kazlauskas (Kaunas University of Technology)

Test solvers

- Björn Martinsson (KTH Royal Institute of Technology)
- Daniel Nieto (Universitat de València)
- Nardos Wehabe (Addis Ababa Science and Technology University)

A: Arithmetic Adaptation

Problem

Given three-digit integer s , find nonzero three-digit integers a and b with $a + b = s$.

Almost-solution

One way of (almost) doing it:

- Set $a = s + 1$, $b = -1$.

A: Arithmetic Adaptation

Problem

Given three-digit integer s , find nonzero three-digit integers a and b with $a + b = s$.

Almost-solution

One way of (almost) doing it:

- Set $a = s + 1$, $b = -1$.
- **Edge cases:** This works except for two cases: when $s = 999$ then $a = 1000$ (which is forbidden), and when $s = -1$ then $a = 0$ (which is also forbidden). These two inputs need special attention (and can again be solved in many ways.)

A: Arithmetic Adaptation

Full solutions

- **Intended:**

$$(a, b) = \begin{cases} (1, -2), & \text{if } s = -1; \\ (998, 1) & \text{if } s = 999; \\ (s + 1, -1) & \text{otherwise.} \end{cases}$$

Statistics at 4-hour mark: 491 submissions, 228 accepted, first after 00:01

A: Arithmetic Adaptation

Full solutions

- **Intended:**

$$(a, b) = \begin{cases} (1, -2), & \text{if } s = -1; \\ (998, 1) & \text{if } s = 999; \\ (s + 1, -1) & \text{otherwise.} \end{cases}$$

- **Using randomness:** Pick random $a \neq s$ of the same sign as s , *i.e.*,

$$a \in \begin{cases} \{-999, \dots, -1\} - \{s\}, & \text{if } s < 0; \\ \{1, \dots, 999\} - \{s\}, & \text{if } s \geq 0; \end{cases}$$

and set $b = s - a$.

A: Arithmetic Adaptation

Full solutions

- **Intended:**

$$(a, b) = \begin{cases} (1, -2), & \text{if } s = -1; \\ (998, 1) & \text{if } s = 999; \\ (s + 1, -1) & \text{otherwise.} \end{cases}$$

- **Using randomness:** Pick random $a \neq s$ of the same sign as s , i.e.,

$$a \in \begin{cases} \{-999, \dots, -1\} - \{s\}, & \text{if } s < 0; \\ \{1, \dots, 999\} - \{s\}, & \text{if } s \geq 0; \end{cases}$$

and set $b = s - a$.

- **Brute force:** Test *all* $a, b \in \{-999, \dots, 999\} - \{0\}$ and check if $a + b = s$. Only four million cases, so this is still fast enough.

Statistics at 4-hour mark: 491 submissions, 228 accepted, first after 00:01

K: km/h

Problem

Given N road signs $s_1, \dots, s_N \in \{10, \dots, 160\} \cup \{\emptyset\}$, determine for each i a maximal speed that is legal based on the first i signs. There is an (unknown) $L \in \{20, 30, 40, \dots\}$ such that $s_i < L$, which comes into effect at each $\emptyset$.

K: km/h

Solution

- **Speed limit sign:** If $s_i \neq \emptyset$ then the i th answer is just s_i .

Solution

- **Speed limit sign:** If $s_i \neq \emptyset$ then the i th answer is just s_i .
- **Derestriction sign:** If $s_i = \emptyset$ find smallest speed $L' \in \{20, 30, 40, \dots\}$ for which $s_j < L'$ for all $j \leq i$. Compute max limit seen so far:

$$M = \max_{1 \leq j \leq i: s_j \neq \emptyset} s_j$$

and find the smallest number $L' > M$ that is a multiple of 10. This can be found either by increasing $M + 1, M + 2, \dots$ until you find a multiple of 10 or as

$$L' = \left(\left\lfloor \frac{M}{10} \right\rfloor + 1 \right) \cdot 10.$$

Solution

- **Speed limit sign:** If $s_i \neq \emptyset$ then the i th answer is just s_i .
- **Derestriction sign:** If $s_i = \emptyset$ find smallest speed $L' \in \{20, 30, 40, \dots\}$ for which $s_j < L'$ for all $j \leq i$. Compute max limit seen so far:

$$M = \max_{1 \leq j \leq i: s_j \neq \emptyset} s_j$$

and find the smallest number $L' > M$ that is a multiple of 10. This can be found either by increasing $M + 1, M + 2, \dots$ until you find a multiple of 10 or as

$$L' = \left(\left\lfloor \frac{M}{10} \right\rfloor + 1 \right) \cdot 10.$$

- **Time:** Computing M from scratch for each i takes $O(i)$, for a total of $O(N^2)$ time. Fast enough because N is tiny. Can also be done in $O(N)$.

C: Crochet Competition

Problem

Given an interval described by two timestamps s and f within the same week, compute their difference. Each timestamp is described by a weekday and a clock time (e.g., Mon 10:27). The difference must be reported as nonzero integer days, hours, minutes; with precise formatting between components.

C: Crochet Competition

Problem

Given an interval described by two timestamps s and f within the same week, compute their difference. Each timestamp is described by a weekday and a clock time (e.g., Mon 10:27). The difference must be reported as nonzero integer days, hours, minutes; with precise formatting between components.

Timestamp to minutes:

Express each time stamp in terms of full minutes since Monday at 00:00: Associate each day name with the respective duration in minutes, e.g.,

$$\text{Mon} \mapsto 0 \cdot 24 \cdot 60, \text{Tue} \mapsto 1 \cdot 24 \cdot 60, \dots, \text{Sun} \mapsto 6 \cdot 24 \cdot 60.$$

Add the clock time HH:MM, translated to full minutes ($\text{HH} \cdot 60 + \text{MM}$).

C: Crochet Competition

Compute difference

- Subtract s (translated to minutes) from f (translated to minutes).

Statistics at 4-hour mark: 650 submissions, 193 accepted, first after 00:10

C: Crochet Competition

Compute difference

- Subtract s (translated to minutes) from f (translated to minutes).
- **Pitfall 1.** If the difference is negative, the interval “wraps around” Mon 00:00. In that case, add $7 \cdot 24 \cdot 60$ (a full week).

Statistics at 4-hour mark: 650 submissions, 193 accepted, first after 00:10

C: Crochet Competition

Compute difference

- Subtract s (translated to minutes) from f (translated to minutes).
- **Pitfall 1.** If the difference is negative, the interval “wraps around” Mon 00:00. In that case, add $7 \cdot 24 \cdot 60$ (a full week).
- **Pitfall 2.** If the difference is 0, the interval is a full week and the output is “7 days”.

Statistics at 4-hour mark: 650 submissions, 193 accepted, first after 00:10

C: Crochet Competition

Compute difference

- Subtract s (translated to minutes) from f (translated to minutes).
- **Pitfall 1.** If the difference is negative, the interval “wraps around” Mon 00:00. In that case, add $7 \cdot 24 \cdot 60$ (a full week).
- **Pitfall 2.** If the difference is 0, the interval is a full week and the output is “7 days”.
- **Output.** In general, the resulting difference t (in minutes) is translated back to the number d of days, h of hours, and m of minutes; most elegantly using integer division with remainder. Finally, report nonzero d , h , and m correctly formatted.

Statistics at 4-hour mark: 650 submissions, 193 accepted, first after 00:10

I: Instagraph

Problem

Given a directed graph G , determine the lowest-indexed vertex v maximising the number of in-neighbours that are not v 's out-neighbours.

Observations

- *Almost* 'just' a question of finding the vertex of highest in-degree. (For this, iterate over all $uv \in E$ and increase indegree counter at v .)

Solution

I: Instagraph

Problem

Given a directed graph G , determine the lowest-indexed vertex v maximising the number of in-neighbours that are not v 's out-neighbours.

Observations

- *Almost* 'just' a question of finding the vertex of highest in-degree. (For this, iterate over all $uv \in E$ and increase indegree counter at v .)
- But need to disregard 'antiparallel' edges, i.e., when both $uv \in E$ and $vu \in E$.

Solution

I: Instagraph

Problem

Given a directed graph G , determine the lowest-indexed vertex v maximising the number of in-neighbours that are not v 's out-neighbours.

Observations

- *Almost* 'just' a question of finding the vertex of highest in-degree. (For this, iterate over all $uv \in E$ and increase indegree counter at v .)
- But need to disregard 'antiparallel' edges, i.e., when both $uv \in E$ and $vu \in E$.

Solution

- Construct the set $\overleftarrow{E} = \{vu : uv \in E\}$ of reverse edges. Then find highest indegree vertex in of $G' = (V, E - \overleftarrow{E})$. Time $O(|E| + |V|)$.

I: Instagraph

Problem

Given a directed graph G , determine the lowest-indexed vertex v maximising the number of in-neighbours that are not v 's out-neighbours.

Observations

- *Almost* 'just' a question of finding the vertex of highest in-degree. (For this, iterate over all $uv \in E$ and increase indegree counter at v .)
- But need to disregard 'antiparallel' edges, i.e., when both $uv \in E$ and $vu \in E$.

Solution

- Construct the set $\overleftarrow{E} = \{vu : uv \in E\}$ of reverse edges. Then find highest indegree vertex in of $G' = (V, E - \overleftarrow{E})$. Time $O(|E| + |V|)$.
- Many other ways, such as storing incoming edges around each v as a set and (in a second pass) removing uv around v when handling vu .

G: Gotta Trade Some of 'em

Problem

Given a n children, and m friendships, determine which of k game variants to assign to each child, so they can use trades between friends to ensure everyone is able to get all Pokémon from all variants.

Translation and solution idea

Given an undirected graph with n nodes and m edges, provide a k -coloring such that each connected component has at least one of each color. Only possible if each connected component is of size at least k . When assigning colors, better to assign color that has not been assigned already to a node in same component.

G: Gotta Trade Some of 'em

Solutions

- Use a depth first search to greedily assign colors. On the i -th node you visit mark it with color $i \pmod k$.
- Alternatively, use a breadth first search. Time complexity $\mathcal{O}(n + m)$
- Alternatively, use the Union-Find data structure with sets to track colors used in each component. Small-to-large merging ensures $\mathcal{O}(n \log^2 n + m)$ time complexity.
- Alternatively, use linked lists in the Union-Find data structure to track which nodes belong to which component. Then use same greedy strategy as above. Time complexity $\mathcal{O}(n\alpha(n) + m)$ if path compression is used.

Statistics at 4-hour mark: 343 submissions, 128 accepted, first after 00:19

Problem

There are N points in the plane $q_1, q_2, \dots, q_N$ that satisfy the **dune dash property**: for every triple $i < j < k$,

$$\text{dist}(q_i, q_k) > \max(\text{dist}(q_i, q_j), \text{dist}(q_j, q_k)).$$

You are given the points, but not in the correct order, and your task is to calculate

$$\text{dist}(q_1, q_2) + \text{dist}(q_2, q_3) + \dots + \text{dist}(q_{N-1}, q_N).$$

Main observation

- **Observation:** The distances from q_1 are increasing:

$$\text{dist}(q_1, q_2) < \text{dist}(q_1, q_3) < \cdots < \text{dist}(q_1, q_N)$$

Main observation

- **Observation:** The distances from q_1 are increasing:

$$\text{dist}(q_1, q_2) < \text{dist}(q_1, q_3) < \cdots < \text{dist}(q_1, q_N)$$

- More generally, for any index i , we have that

$$\text{dist}(q_i, q_{i+1}) < \text{dist}(q_i, q_{i+2}) < \cdots < \text{dist}(q_i, q_N)$$

and

$$\text{dist}(q_i, q_{i-1}) < \text{dist}(q_i, q_{i-2}) < \cdots < \text{dist}(q_i, q_1)$$

Solution

- Take an arbitrary point p from the input, and find the point furthest away from it. This must be one of the endpoints q_1 or q_N .

Solution

- Take an arbitrary point p from the input, and find the point furthest away from it. This must be one of the endpoints q_1 or q_N .
- Sort the remaining points by distance from the endpoint. This will be the correct ordering of the points. Finally, add up the distances.

Solution

- Take an arbitrary point p from the input, and find the point furthest away from it. This must be one of the endpoints q_1 or q_N .
- Sort the remaining points by distance from the endpoint. This will be the correct ordering of the points. Finally, add up the distances.
- $O(N \log N)$

Alternative solutions

- **Solution 2:** The dune dash property implies that the line graph is a minimum spanning tree. Euclidean MST can be solved in $O(N \log N)$.

Statistics at 4-hour mark: 166 submissions, 45 accepted, first after 00:44

Alternative solutions

- **Solution 2:** The dune dash property implies that the line graph is a minimum spanning tree. Euclidean MST can be solved in $O(N \log N)$.
- **Solution 3:** The two endpoints are the two most distant points in the given set. These can be found with convex hull and rotating calipers. Then the points can be sorted by distance.

Statistics at 4-hour mark: 166 submissions, 45 accepted, first after 00:44

Alternative solutions

- **Solution 2:** The dune dash property implies that the line graph is a minimum spanning tree. Euclidean MST can be solved in $O(N \log N)$.
- **Solution 3:** The two endpoints are the two most distant points in the given set. These can be found with convex hull and rotating calipers. Then the points can be sorted by distance.
- **Solution 4:** Any subset of the given points also satisfies the dune dash property. This allows for various divide-and-conquer solutions.

Statistics at 4-hour mark: 166 submissions, 45 accepted, first after 00:44

Alternative solutions

- **Solution 2:** The dune dash property implies that the line graph is a minimum spanning tree. Euclidean MST can be solved in $O(N \log N)$.
- **Solution 3:** The two endpoints are the two most distant points in the given set. These can be found with convex hull and rotating calipers. Then the points can be sorted by distance.
- **Solution 4:** Any subset of the given points also satisfies the dune dash property. This allows for various divide-and-conquer solutions.
- **Bonus problem:** Can you check if a given set of points satisfies the dune dash property? Same constraints as the original problem.

Statistics at 4-hour mark: 166 submissions, 45 accepted, first after 00:44

E: Egyptian Equality

Problem

Given a pyramidal grid and a subset C of *casing* grid cells, partition the grid into sideways connected components A and B such that $|A \cap C| = |B \cap C|$. (Or report that this cannot be done.)

E: Egyptian Equality

Rectangular case

- If not for the pyramidal shape, this would be an easy problem that can be solved “greedily”.

			C	C				
			C					
		C						
C			C	C		C		
	C	C			C			C

E: Egyptian Equality

Rectangular case

- If not for the pyramidal shape, this would be an easy problem that can be solved “greedily”.

			C	C				
			C					
		C						
C			C	C		C		
	C	C			C			C

- **Rectangular solution:** Traverse the cells in some systematic, connected way, for instance in ‘reading order’ left-to right, top-down. Put all cells into A until you have seen $\frac{1}{2}|C|$ many C . Then put the remaining cells into B .

E: Egyptian Equality

Rectangular case

- If not for the pyramidal shape, this would be an easy problem that can be solved “greedily”.

			C	C				
			C					
		C						
C			C	C		C		
	C	C			C			C

- **Rectangular solution:** Traverse the cells in some systematic, connected way, for instance in ‘reading order’ left-to right, top-down. Put all cells into A until you have seen $\frac{1}{2}|C|$ many C . Then put the remaining cells into B .

			C	C				
			C					
		C						
C			C	C		C		
	C	C			C			C

E: Egyptian Equality

Rectangular case

- If not for the pyramidal shape, this would be an easy problem that can be solved “greedily”.

			C	C				
			C					
		C						
C			C	C		C		
	C	C			C			C

- Rectangular solution:** Traverse the cells in some systematic, connected way, for instance in ‘reading order’ left-to right, top-down. Put all cells into A until you have seen $\frac{1}{2}|C|$ many C . Then put the remaining cells into B .

			C	C				
			C					
		C						
C			C	C		C		
	C	C			C			C

			C	C				
			C					
		C						
C			C	C		C		
	C	C			C			C

E: Egyptian Equality

Rectangular case

- If not for the pyramidal shape, this would be an easy problem that can be solved “greedily”.

			C	C				
			C					
		C						
C			C	C		C		
	C	C			C			C

- **Rectangular solution:** Traverse the cells in some systematic, connected way, for instance in ‘reading order’ left-to right, top-down. Put all cells into A until you have seen $\frac{1}{2}|C|$ many C . Then put the remaining cells into B .

			C	C				
			C					
		C						
C			C	C		C		
	C	C			C			C

			C	C				
			C					
		C						
C			C	C		C		
	C	C			C			C

			C	C				
			C					
		C						
C			C	C		C		
	C	C			C			C

E: Egyptian Equality

Why Greedy Does not Work in the Pyramid.

- Greedy (say, from the bottom) gets stuck in a corner after assigning the first 4 Cs to A . Now, region B cannot be connected.



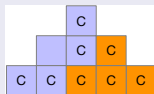
E: Egyptian Equality

Why Greedy Does not Work in the Pyramid.

- Greedy (say, from the bottom) gets stuck in a corner after assigning the first 4 Cs to *A*. Now, region *B* cannot be connected.



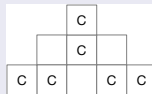
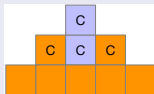
- The solution to this instance looks quite different:



E: Egyptian Equality

Other Instructive Edge Cases.

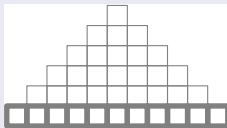
The solution to the left case is hard to find using most cell orders. The right case has no solution at all.



E: Egyptian Equality

Solution Case 1: Sparse Bottom Row

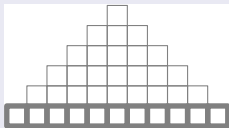
- **Bottom row.** Let B be the cells at the bottom.



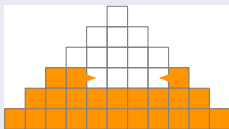
E: Egyptian Equality

Solution Case 1: Sparse Bottom Row

- **Bottom row.** Let B be the cells at the bottom.



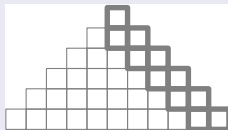
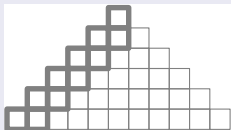
- **Greedy works.** If $|C \cap B| \leq \frac{1}{2}|C|$ then the entire bottom row can be assigned to A . After that, we can greedily “flood” the pyramid, row-by-row, from the outside in, and are guaranteed that A remains connected.



E: Egyptian Equality

Solution Case 2 and 3: Sparse Sides

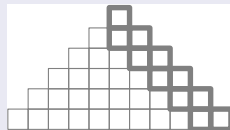
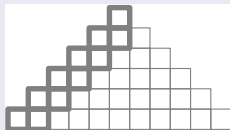
- **Sides.** Let L and R be the two bands of width 2 at the sides



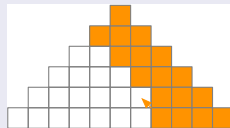
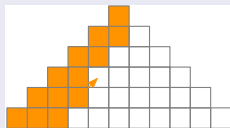
E: Egyptian Equality

Solution Case 2 and 3: Sparse Sides

- **Sides.** Let L and R be the two bands of width 2 at the sides



- **Greedy works.** If $|C \cap L| \leq \frac{1}{2}|C|$ then all of L can be assigned to A . After that, we can greedily traverse the pyramid diagonally and are guaranteed that A remains connected. Symmetrically, if $|C \cap R| \leq \frac{1}{2}|C|$ then all of R can be assigned to A .

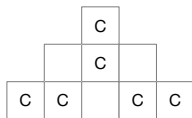


E: Egyptian Equality

- **We're done.** Not all of

$$|C \cap B| \leq \frac{1}{2}|C|, |C \cap L| \leq \frac{1}{2}|C|, |C \cap R| \leq \frac{1}{2}|C|,$$

can fail simultaneously, except for a completely filled pyramid with $|C| = 4$ and the impossible case



Thus, one of the three greedy traversals (flood, diagonal, anti-diagonal) must work.

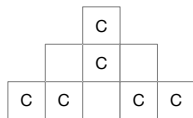
Statistics at 4-hour mark: 219 submissions, 26 accepted, first after 01:07

E: Egyptian Equality

- **We're done.** Not all of

$$|C \cap B| \leq \frac{1}{2}|C|, |C \cap L| \leq \frac{1}{2}|C|, |C \cap R| \leq \frac{1}{2}|C|,$$

can fail simultaneously, except for a completely filled pyramid with $|C| = 4$ and the impossible case



Thus, one of the three greedy traversals (flood, diagonal, anti-diagonal) must work.

- Probably many other approaches work as well.

Statistics at 4-hour mark: 219 submissions, 26 accepted, first after 01:07

B: Bohemian Bookshelf

Problem

Given a $W \times H$ bookshelf and N books with spine height h_i and thickness t_i ($1 \leq i \leq N$), arrange each book either upright or as part of a single stack. The stacked books need to form a nicely conical stack, tallest book at the bottom. There needs to be at least one upright and one stacked book.

Observations

- The hard part is to determine which books are stacked and which are upright; everything else (checking that this partition actually fits, and arranging the stacked books in nondecreasing order) is relatively straightforward.

B: Bohemian Bookshelf

Problem

Given a $W \times H$ bookshelf and N books with spine height h_i and thickness t_i ($1 \leq i \leq N$), arrange each book either upright or as part of a single stack. The stacked books need to form a nicely conical stack, tallest book at the bottom. There needs to be at least one upright and one stacked book.

Observations

- The hard part is to determine which books are stacked and which are upright; everything else (checking that this partition actually fits, and arranging the stacked books in nondecreasing order) is relatively straightforward.
- Naive solution: try all 2^{N-1} nonempty partitions. Correct, but too slow for $N = 100$.

B: Bohemian Bookshelf

Problem

Given a $W \times H$ bookshelf and N books with spine height h_i and thickness t_i ($1 \leq i \leq N$), arrange each book either upright or as part of a single stack. The stacked books need to form a nicely conical stack, tallest book at the bottom. There needs to be at least one upright and one stacked book.

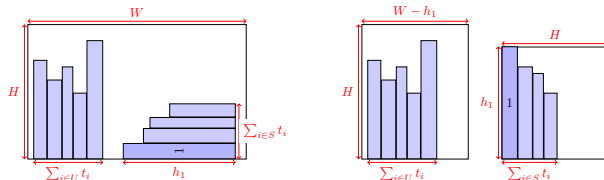
Observations

- The hard part is to determine which books are stacked and which are upright; everything else (checking that this partition actually fits, and arranging the stacked books in nondecreasing order) is relatively straightforward.
- Naive solution: try all 2^{N-1} nonempty partitions. Correct, but too slow for $N = 100$.
- Greedily laying all the large books down and placing all the fat books upright fails.

B: Bohemian Bookshelf

Reduction to Subset Sum

If we assume that book 1 will be the bottommost *stacked* book.



Simplified problem:

Fix $1 \in S$ and partition the remaining books $\{2, \dots, n\}$ into two sets U and S ('upright' and 'stacked'), such that

- the books in U have spine height $\leq H$ and total thickness $\leq W - h_1$.
- the books in S have spine height $\leq h_1$ and total thickness $\leq H$.

Solution

- Next slide: solve simplified problem (with book 1 as “bottommost stacked book”) in time $O(NWH)$.

Solution

- Next slide: solve simplified problem (with book 1 as “bottommost stacked book”) in time $O(NWH)$.
- Iterate over all candidates for “bottommost stacked book”; total time $O(N^2WH)$.

Solution

- Next slide: solve simplified problem (with book 1 as “bottommost stacked book”) in time $O(NWH)$.
- Iterate over all candidates for “bottommost stacked book”; total time $O(N^2WH)$.
- More careful analysis gives $O(N(W + H))$ for simplified problem. More ideas can reduce time for the full problem to $O(N(W + H))$, but not needed for this task.

B: Bohemian Bookshelf

Solution to Simplified Problem (Dynamic Programming)

- For $i \in \{1, \dots, n\}$ and total thicknesses $T_u \in \{0, \dots, H\}$, $T_s \in \{0, \dots, W\}$ let $p(i, T_u, T_s)$ be a packing S of the first i books such that $1 \in S$ and

$$T_u = \sum_{j \notin S} t_j, \quad T_s = \sum_{j \in S} t_j.$$

Solution to Simplified Problem (Dynamic Programming)

- For $i \in \{1, \dots, n\}$ and total thicknesses $T_u \in \{0, \dots, H\}$, $T_s \in \{0, \dots, W\}$ let $p(i, T_u, T_s)$ be a packing S of the first i books such that $1 \in S$ and

$$T_u = \sum_{j \notin S} t_j, \quad T_s = \sum_{j \in S} t_j.$$

- (Base case.) Then $p(1, T_u, T_s)$ is true if and only if $T_u = 0$ and $T_s = t_1$.

Solution to Simplified Problem (Dynamic Programming)

- For $i \in \{1, \dots, n\}$ and total thicknesses $T_u \in \{0, \dots, H\}$, $T_s \in \{0, \dots, W\}$ let $p(i, T_u, T_s)$ be a packing S of the first i books such that $1 \in S$ and

$$T_u = \sum_{j \notin S} t_j, \quad T_s = \sum_{j \in S} t_j.$$

- (Base case.) Then $p(1, T_u, T_s)$ is true if and only if $T_u = 0$ and $T_s = t_1$.
- (Recurrence.) Moreover, p satisfies the recurrence for $i \in \{2, \dots, n\}$,

$$p(i, T_u, T_s) = \bigcup \begin{cases} p(i-1, T_u - t_i, T_s), & \text{if } h_i \leq H \text{ (shelve } i\text{th book upright);} \\ p(i-1, T_u, T_s - t_i), & \text{if } h_i \leq h_1 \text{ (stack } i\text{th book).} \end{cases}$$

B: Bohemian Bookshelf

Solution to Simplified Problem (Dynamic Programming)

- For $i \in \{1, \dots, n\}$ and total thicknesses $T_u \in \{0, \dots, H\}$, $T_s \in \{0, \dots, W\}$ let $p(i, T_u, T_s)$ be a packing S of the first i books such that $1 \in S$ and

$$T_u = \sum_{j \notin S} t_j, \quad T_s = \sum_{j \in S} t_j.$$

- (Base case.) Then $p(1, T_u, T_s)$ is true if and only if $T_u = 0$ and $T_s = t_1$.
- (Recurrence.) Moreover, p satisfies the recurrence for $i \in \{2, \dots, n\}$,

$$p(i, T_u, T_s) = \bigcup \begin{cases} p(i-1, T_u - t_i, T_s), & \text{if } h_i \leq H \text{ (shelve } i\text{th book upright);} \\ p(i-1, T_u, T_s - t_i), & \text{if } h_i \leq h_1 \text{ (stack } i\text{th book).} \end{cases}$$

- Recurrence solved using dynamic programming in time $O(NWH)$.

Statistics at 4-hour mark: 150 submissions, 20 accepted, first after 01:10

Problem

Simulate the following process starting with E energy and starting sum $S = 0$.
Increase S by E , if $S = 0 \pmod n$ increase E by one, otherwise decrease E by 1.
If E becomes 0, stop.
Report what the final sum is, or report that it keeps going forever.

Solution

- We can consider states of the form $(S \pmod n, E \pmod n)$.
- These states determine uniquely which other state we go to next, though we will have to make sure to use the full value of E rather than $E \pmod n$ when we calculate the final answer.
- But these are $\mathcal{O}(n^2)$ states, so we still need something more to reduce this.

Solution

- Any time our first coordinate, $S \pmod n$, is non-zero our next state will have E one lower.
- So we will go from (S, E) to $(S + E, E - 1)$, to $(S + E + E - 1, E - 2)$ and so on until the first coordinate is zero.
- Thus if we can determine how many steps it will take for the first coordinate to become zero we can simulate all those steps in one go.
- So for each $S \pmod n$ we would like to find the minimal $f(S)$ such that $S + f(S) + (f(S) - 1) + \cdots + 2 + 1 = 0 \pmod n$
- We can rewrite this as $S + f(S)(f(S) + 1)/2 = 0 \pmod n$

Solution

- Solving $S + f(S)(f(S) + 1)/2 = 0 \pmod{n}$ can be done several ways, some do not require finding modular square roots.
- One way is to iterate over the possible values of $1 \leq f(S) \leq 2n$ and for each of those divide into cases based on parity. Then divide through by the gcd to get a linear equation for $f(S)$ modulo some d dividing n .
- Then that solution can be applied to all multiples of d , thus getting $f(S)$ for all S .
- But there are many ways to get $f(S)$ for all n relevant values of S .

Solution

- Now we only consider states of the form $(0, E \pmod n)$, so there are n states.
- We find cycles in $\mathcal{O}(n)$ and check if the energy gain is non-negative for a full cycle and possibly report infinity.
- Otherwise the distance traveled per cycle will form an arithmetic sequence which can be summed to get the final answer.

Solution

- Open problem: prove that the cycles have length $O(n)$.
- This means as far as we know the much simpler solution of just checking the cycles and skipping all the business with $f(S)$ works fine.
- Bonus problem: Solve for $n \leq 10^9$.

Statistics at 4-hour mark: 78 submissions, 5 accepted, first after 01:54

Problem

Given n accounts on a social media platform, where each account follows a_i accounts and is followed by b_i accounts, construct a network such that it is fully viral. A network is fully viral if any post made by an account can reach any other account through a chain of shares and comments.

Translation

Construct a weakly-connected directed graph with the given out-degrees and in-degrees.

Starting simple

- Lets simplify and consider a degree sequence $d_1, d_2, \dots, d_n$ for an undirected graph.
- A natural approach is just to pick some vertex i and connect to d_i other nodes.
- Impossible if any degree $d_i > n - 1$. Also impossible if sum of degrees is odd.

Starting simple

- Lets simplify and consider a degree sequence $d_1, d_2, \dots, d_n$ for an undirected graph.
- A natural approach is just to pick some vertex i and connect to d_i other nodes.
- Impossible if any degree $d_i > n - 1$. Also impossible if sum of degrees is odd.
- Which vertices to connect to?

Starting simple

- Lets simplify and consider a degree sequence $d_1, d_2, \dots, d_n$ for an undirected graph.
- A natural approach is just to pick some vertex i and connect to d_i other nodes.
- Impossible if any degree $d_i > n - 1$. Also impossible if sum of degrees is odd.
- Which vertices to connect to?
- Need to keep maximum degree low, so connect to largest degrees.
- Natural to select largest vertex as the hub too, but turns out you can select any vertex as hub.
- Repeat this hub selection and update sequence until sequence is all zeros.

Starting simple

- Lets simplify and consider a degree sequence $d_1, d_2, \dots, d_n$ for an undirected graph.
- A natural approach is just to pick some vertex i and connect to d_i other nodes.
- Impossible if any degree $d_i > n - 1$. Also impossible if sum of degrees is odd.
- Which vertices to connect to?
- Need to keep maximum degree low, so connect to largest degrees.
- Natural to select largest vertex as the hub too, but turns out you can select any vertex as hub.
- Repeat this hub selection and update sequence until sequence is all zeros.
- This is the “well known” Havel-Hakimi algorithm.

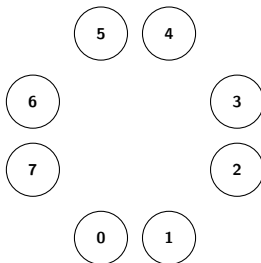
Example degree sequence

index	0	1	2	3	4	5	6	7
original	1	1	1	1	2	2	3	3

F: Follower Forensics

Example degree sequence

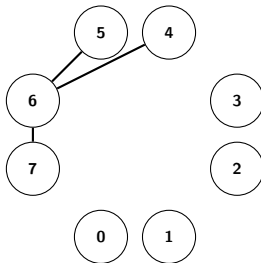
index	0	1	2	3	4	5	6	7
original	1	1	1	1	2	2	3	3
current	1	1	1	1	2	2	3	3



F: Follower Forensics

Example degree sequence

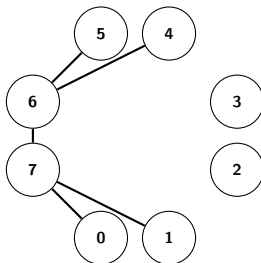
index	0	1	2	3	4	5	6	7
original	1	1	1	1	2	2	3	3
current	1	1	1	1	1	1	0	2



F: Follower Forensics

Example degree sequence

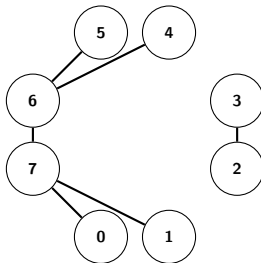
index	0	1	2	3	4	5	6	7
original	1	1	1	1	2	2	3	3
current	0	0	1	1	1	1	0	0



F: Follower Forensics

Example degree sequence

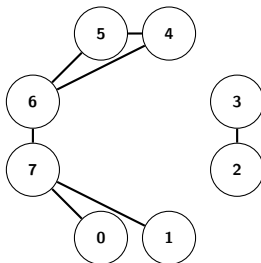
index	0	1	2	3	4	5	6	7
original	1	1	1	1	2	2	3	3
current	0	0	0	0	1	1	0	0



F: Follower Forensics

Example degree sequence

index	0	1	2	3	4	5	6	7
original	1	1	1	1	2	2	3	3
current	0	0	0	0	0	0	0	0



Staying connected

- Now d_i must be at least 1 for each vertex and we need at least $n - 1$ edges total.
- Picking a large degree vertex as hub connects large degrees to large degrees.

Staying connected

- Now d_i must be at least 1 for each vertex and we need at least $n - 1$ edges total.
- Picking a large degree vertex as hub connects large degrees to large degrees.
- This means you can form many small components.
- By picking a small degree as a hub, we ensure it gets connected to a large degree.

Staying connected

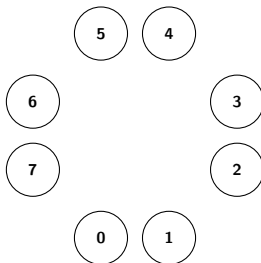
- Now d_i must be at least 1 for each vertex and we need at least $n - 1$ edges total.
- Picking a large degree vertex as hub connects large degrees to large degrees.
- This means you can form many small components.
- By picking a small degree as a hub, we ensure it gets connected to a large degree.
- This maximizes the number of times we connect two separate connected components as we avoid making cycles when possible.
- Acyclic for $m \leq n - 1$ and connected for $m \geq n - 1$.

Example degree sequence

index	0	1	2	3	4	5	6	7
original	1	1	1	1	2	2	3	3
current	1	1	1	1	2	2	3	3

Example degree sequence

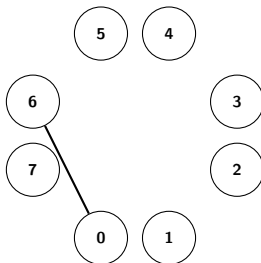
index	0	1	2	3	4	5	6	7
original	1	1	1	1	2	2	3	3
current	1	1	1	1	2	2	3	3



F: Follower Forensics

Example degree sequence

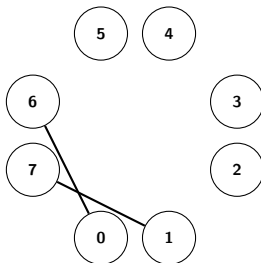
index	0	1	2	3	4	5	6	7
original	1	1	1	1	2	2	3	3
current	0	1	1	1	2	2	2	3



F: Follower Forensics

Example degree sequence

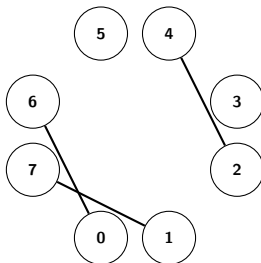
index	0	1	2	3	4	5	6	7
original	1	1	1	1	2	2	3	3
current	0	0	1	1	2	2	2	2



F: Follower Forensics

Example degree sequence

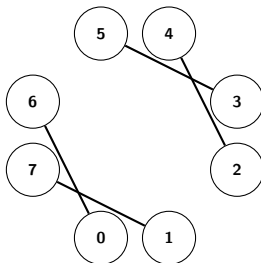
index	0	1	2	3	4	5	6	7
original	1	1	1	1	2	2	3	3
current	0	0	0	1	1	2	2	2



F: Follower Forensics

Example degree sequence

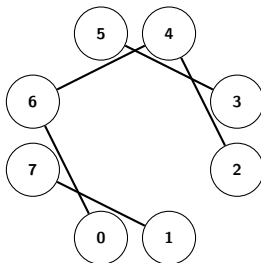
index	0	1	2	3	4	5	6	7
original	1	1	1	1	2	2	3	3
current	0	0	0	0	1	1	2	2



F: Follower Forensics

Example degree sequence

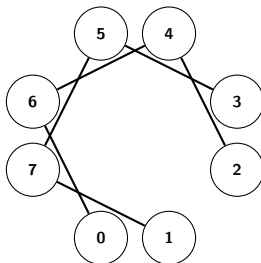
index	0	1	2	3	4	5	6	7
original	1	1	1	1	2	2	3	3
current	0	0	0	0	0	1	1	2



F: Follower Forensics

Example degree sequence

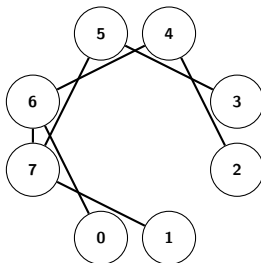
index	0	1	2	3	4	5	6	7
original	1	1	1	1	2	2	3	3
current	0	0	0	0	0	0	1	1



F: Follower Forensics

Example degree sequence

index	0	1	2	3	4	5	6	7
original	1	1	1	1	2	2	3	3
current	0	0	0	0	0	0	0	0



Adding direction

- We have two sequences for directed graphs.
- Can choose a hub and either use up its in-degree or its out-degree.
- Algorithm remains the same, but largest degree vertex becomes a bit ambiguous.
- If we use up the in-degree of the hub, then order by out-degree and break ties on in-degree, and vice-versa.

Solution

- Apply the same idea as with undirected graphs, picking the smallest degree.
- Find the lowest in-degree and lowest out-degree, pick whichever is lower.
- Connect to largest out-degrees or largest in-degrees depending on previous choice.
- If graph is weakly-connected at the end, then output it.
- Otherwise, output impossible.
- Many ways to implement, simplest fast way is $\mathcal{O}(m \log n)$ using ordered sets.

Statistics at 4-hour mark: 18 submissions, 1 accepted, first after 03:03

Solution

- Apply the same idea as with undirected graphs, picking the smallest degree.
- Find the lowest in-degree and lowest out-degree, pick whichever is lower.
- Connect to largest out-degrees or largest in-degrees depending on previous choice.
- If graph is weakly-connected at the end, then output it.
- Otherwise, output impossible.
- Many ways to implement, simplest fast way is $\mathcal{O}(m \log n)$ using ordered sets.
- Bonus Challenge: Design and implement a linear algorithm...

Statistics at 4-hour mark: 18 submissions, 1 accepted, first after 03:03

Solution

- Apply the same idea as with undirected graphs, picking the smallest degree.
- Find the lowest in-degree and lowest out-degree, pick whichever is lower.
- Connect to largest out-degrees or largest in-degrees depending on previous choice.
- If graph is weakly-connected at the end, then output it.
- Otherwise, output impossible.
- Many ways to implement, simplest fast way is $\mathcal{O}(m \log n)$ using ordered sets.
- Bonus Challenge: Design and implement a linear algorithm...
- ...or read about it and the proofs in Arnar's upcoming paper.

Statistics at 4-hour mark: 18 submissions, 1 accepted, first after 03:03

Solution

- Apply the same idea as with undirected graphs, picking the smallest degree.
- Find the lowest in-degree and lowest out-degree, pick whichever is lower.
- Connect to largest out-degrees or largest in-degrees depending on previous choice.
- If graph is weakly-connected at the end, then output it.
- Otherwise, output impossible.
- Many ways to implement, simplest fast way is $\mathcal{O}(m \log n)$ using ordered sets.
- Bonus Challenge: Design and implement a linear algorithm...
- ...or read about it and the proofs in Arnar's upcoming paper.
- Super Duper Bonus Challenge: Strongly connected graphs in linear time...

Statistics at 4-hour mark: 18 submissions, 1 accepted, first after 03:03

Solution

- Apply the same idea as with undirected graphs, picking the smallest degree.
- Find the lowest in-degree and lowest out-degree, pick whichever is lower.
- Connect to largest out-degrees or largest in-degrees depending on previous choice.
- If graph is weakly-connected at the end, then output it.
- Otherwise, output impossible.
- Many ways to implement, simplest fast way is $\mathcal{O}(m \log n)$ using ordered sets.
- Bonus Challenge: Design and implement a linear algorithm...
- ...or read about it and the proofs in Arnar's upcoming paper.
- Super Duper Bonus Challenge: Strongly connected graphs in linear time...
- ...and then call me and tell me how.

Statistics at 4-hour mark: 18 submissions, 1 accepted, first after 03:03

H: Hidden Permutation

Problem

There is a hidden permutation f . You are given K numbers $p_1, p_2, \dots, p_K$ and $m_1, m_2, \dots, m_K$, meaning that there are m_i binary strings with period p_i . Your task is to find the permutation.

H: Hidden Permutation

Some observations

- The only thing that matters are the cycle lengths $c_1, c_2, \dots, c_M$ of the permutation.

H: Hidden Permutation

Some observations

- The only thing that matters are the cycle lengths $c_1, c_2, \dots, c_M$ of the permutation.
- Let a be a positive integer. Define y_a to be the number of binary strings such that $f^a(b) = b$. Then

$$y_a = \sum_{p_b|a} m_b$$

H: Hidden Permutation

Some observations

- The only thing that matters are the cycle lengths $c_1, c_2, \dots, c_M$ of the permutation.
- Let a be a positive integer. Define y_a to be the number of binary strings such that $f^a(b) = b$. Then

$$y_a = \sum_{p_b|a} m_b$$

- But in terms of the cycle lengths, y_a can also be written as

$$y_a = \prod_{j=1}^M 2^{\gcd(c_j, a)} = 2^{\sum_{j=1}^M \gcd(c_j, a)}$$

H: Hidden Permutation

Some observations part 2

- Let d_i be the number of cycles of length i in the permutation f .

H: Hidden Permutation

Some observations part 2

- Let d_i be the number of cycles of length i in the permutation f .
- From the previous slide, we got that

$$\log_2(y_a) = \sum_{j=1}^M \gcd(c_j, a) = \sum_{i=1}^N \gcd(i, a) d_i$$

H: Hidden Permutation

Some observations part 2

- Let d_i be the number of cycles of length i in the permutation f .
- From the previous slide, we got that

$$\log_2(y_a) = \sum_{j=1}^M \gcd(c_j, a) = \sum_{i=1}^N \gcd(i, a) d_i$$

- This means that for every positive integer a , we get a linear equation with d_i as unknowns and $\gcd(i, a)$ as coefficients. We can calculate the numbers y_a from the input, and if we have the numbers d_i , then we can construct the permutation.

H: Hidden Permutation

Some observations part 2

- Let d_i be the number of cycles of length i in the permutation f .
- From the previous slide, we got that

$$\log_2(y_a) = \sum_{j=1}^M \gcd(c_j, a) = \sum_{i=1}^N \gcd(i, a) d_i$$

- This means that for every positive integer a , we get a linear equation with d_i as unknowns and $\gcd(i, a)$ as coefficients. We can calculate the numbers y_a from the input, and if we have the numbers d_i , then we can construct the permutation.
- Let's consider the numbers $a = 1, 2, 3, \dots, N$ and solve the resulting system of equations!

H: Hidden Permutation

Solution

- Compute the numbers $\log_2(y_a)$ via $y_a = \sum_{p_b|a} m_b$, for every $a = 1, 2, 3, \dots, N$

H: Hidden Permutation

Solution

- Compute the numbers $\log_2(y_a)$ via $y_a = \sum_{p_b|a} m_b$, for every $a = 1, 2, 3, \dots, N$
- We now have a linear system of equations

$$\log_2(\mathbf{y}) = G\mathbf{d}$$

where G is the famous **GCD-matrix**:

$$G = \begin{bmatrix} \gcd(1, 1) & \gcd(1, 2) & \gcd(1, 3) & \cdots & \gcd(1, N) \\ \gcd(2, 1) & \gcd(2, 2) & \gcd(2, 3) & \cdots & \gcd(2, N) \\ \gcd(3, 1) & \gcd(3, 2) & \gcd(3, 3) & \cdots & \gcd(3, N) \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \gcd(N, 1) & \gcd(N, 2) & \gcd(N, 3) & \cdots & \gcd(N, N) \end{bmatrix}$$

H: Hidden Permutation

Solution

- The GCD matrix is known to have nonzero determinant, so use Gaussian elimination to solve the system.

Statistics at 4-hour mark: 4 submissions, 0 accepted, first after ???

H: Hidden Permutation

Solution

- The GCD matrix is known to have nonzero determinant, so use Gaussian elimination to solve the system.
- Finally, compute the numbers p_i, m_i again to check that they are the same as in the input.

Statistics at 4-hour mark: 4 submissions, 0 accepted, first after ???

H: Hidden Permutation

Solution

- The GCD matrix is known to have nonzero determinant, so use Gaussian elimination to solve the system.
- Finally, compute the numbers p_i, m_i again to check that they are the same as in the input.
- If you want to avoid big integers or floats: many of the calculations can be done modulo P .

Statistics at 4-hour mark: 4 submissions, 0 accepted, first after ???

Results!